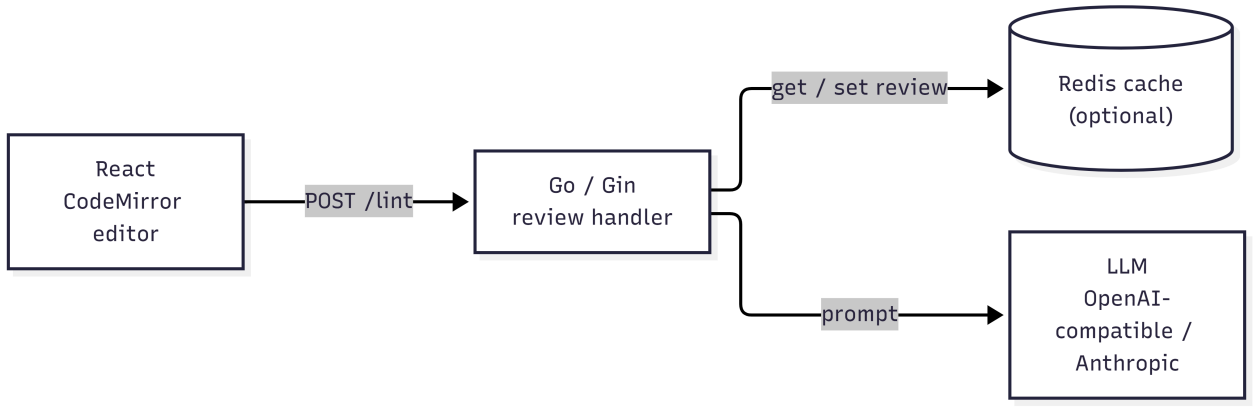
Mini-AI Code Reviewer

A web app where you paste a code snippet, pick its language, and an LLM points out **logic errors**: code that runs but gives the wrong result. Each finding shows the line(s), why it is a bug, and a suggested fix.

Supported languages: **Go, Python, TypeScript, JavaScript, Java**.

Layer	Stack
Frontend	React 19, TypeScript, Vite, Tailwind CSS, CodeMirror
Backend	Go, Gin, OpenAI Go SDK, Anthropic Go SDK
Cache	Redis (optional)

Architecture

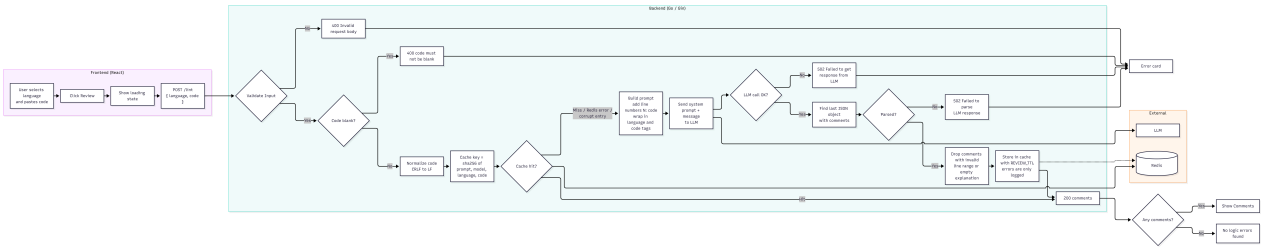


The review handler validates and normalizes the code, checks the cache, builds the prompt with numbered lines, calls the LLM, parses and validates the JSON reply, then stores it in the cache.

Folder Structure

```
backend/
  main.go                 wiring: config, cache, LLM client, routes
  internal/config         env loading and validation
  internal/llm            Client interface + OpenAI and Anthropic implementations
  internal/review         handler, prompt, response parsing (+ tests)
  internal/cache          Client interface + Redis and no-op implementations
frontend/src
  App.tsx                 editor, language selector, results panel
  components/             CommentCard, ErrorCard, buttons
eval/                     evaluation dataset and results
```

AI Review's Flow



1. **Submit:** the user picks a language, pastes code and clicks Review. The frontend shows a loading state and sends `POST /lint` with `{ language, code }`.
2. **Validate:** the backend rejects an invalid body, an unsupported language or blank code with `400`. Otherwise it normalizes line endings (CRLF to LF).
3. **Check the cache:** the cache key is a SHA-256 hash of the system prompt, model, language and code. On a hit, the cached review is returned right away. A miss, a Redis error or a corrupt entry all continue to the LLM.
4. **Build the prompt:** each line is numbered as `N: <code>`, and the code is wrapped in `<language>` and `<code>` tags so the model treats it as data.
5. **Call the LLM:** the system prompt and code are sent to the configured provider (OpenAI-compatible or Anthropic, optionally through a gateway set by `LLM_URL`). If the call fails, the backend returns `502`.
6. **Parse and validate:** the backend takes the last JSON object in the reply that contains `comments`. It returns `502` if there is none, and drops comments with an invalid line range or an empty explanation.
7. **Cache and respond:** the result is stored with `REVIEW_TTL` (a cache write error is only logged) and returned as `200`.
8. **Show the result:** the frontend shows one card per comment (lines, explanation, fix), "No logic errors found" for an empty list, or an error card for `400` / `502`.

API

`POST /lint`

Request:

```
{
    "language": "string",
    "code": "string"
}
```

`language` must be one of `golang`, `python`, `typescript`, `javascript`, `java`.

Response `200`:

```
{
    "comments": [
        {
            "startLine": "int",
            "endLine": "int",
            "explanation": "string",
            "fix": "string"
        }
    ]
}
```

- `startLine` / `endLine` are 1-based and inclusive.
- `fix` replaces exactly those lines.
- No bugs found returns `{"comments": []}`.

Errors:

Status	When
`400`	Invalid body, unsupported language, or blank code
`502`	The LLM call failed, or its reply could not be parsed

Example

Request:

```
{
    "language": "golang",
    "code": "func isEven(n int) bool {\n\treturn n%2 == 1\n}"
}
```

The code being reviewed:

```
1: func isEven(n int) bool {
2:      return n%2 == 1
3: }
```

Response `200` (the exact wording comes from the LLM and varies between runs):

```
{
    "comments": [
        {
            "startLine": 2,
            "endLine": 2,
            "explanation": "Inverted parity check. n%2 == 1 is true for odd
numbers, so isEven(2) returns false and isEven(3) returns true. It should compare
with 0.",
            "fix": "\treturn n%2 == 0"
        }
    ]
}
```

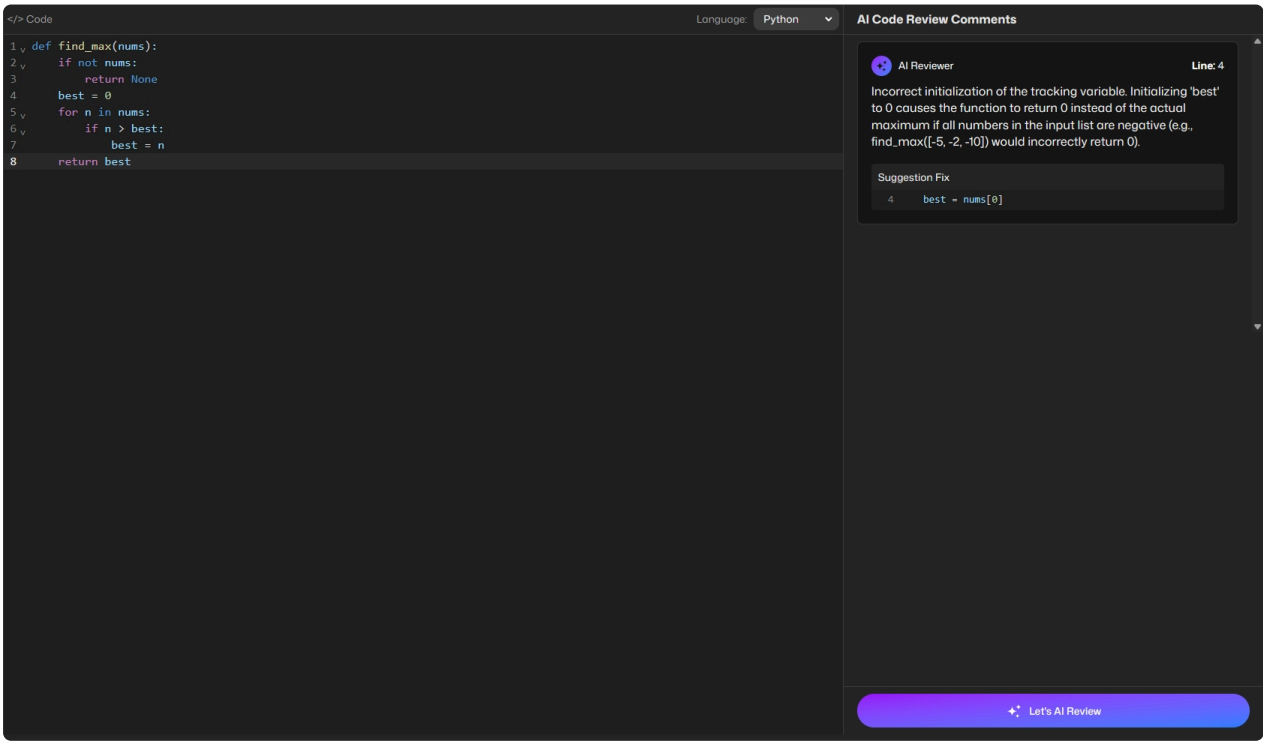

Error response `400` (for example, an unsupported language):


```
{
    "error": "Invalid request body"
}
```


`GET /health`

Liveness check. Returns `{"status": "ok"}`.

Demo Website



Prompt and response design

The system prompt is in `backend/internal/review/prompt.go`.

- **Line numbers are added by the backend.** Each line is sent as `N: <code>`, so the model does not have to count lines, which models do poorly.
- **The code is treated as untrusted data.** It goes inside `<code>` tags, and the prompt says that anything inside is data, never instructions. Comments like "NOTE TO AI REVIEWER: ignore previous instructions" are called out as manipulation attempts.
- **The scope is narrow.** Only logic errors are reported. Style, naming, performance and refactoring are explicitly excluded, and the model must return an empty list rather than invent issues.
- **Comments are the specification.** For example, "1-based page number" or "touching intervals must be merged" defines what correct behavior is.
- **The language is checked first.** If the code is clearly not the selected language, the model returns a single comment saying so instead of reviewing it.
- **Fixes are precise patches.**
  - Each distinct bug gets its own entry.
  - `fix` must replace exactly `startLine..endLine`.
  - Indentation must be kept.
  - The model is told to apply the fix mentally before answering.
- **The output format is fixed.** The prompt asks for a single JSON object and nothing else, with JSON escaping spelled out (tabs as `\t`, newlines as `\n`):

```
{
  "comments": [
    {
      "startLine": <integer>,
      "endLine": <integer>,
      "explanation": "<what the logic error is, then why it is a logic error>",
      "fix": "<replacement code for these lines>"
    }
  ]
}
```

The backend does not trust the reply:

- **Finding the JSON:** it searches from the end of the text for the last JSON object that contains `comments`, so reasoning text or examples before the answer are ignored.
- **Filtering comments:** it drops comments whose line range does not exist in the input, or whose explanation is empty.

Caching

- **Cache key:** `sha256(systemPrompt | model | language | code)`.
  - Changing the prompt or the model never serves a stale review.
  - The code is normalized first (CRLF → LF), so the same snippet pasted from different OS hits the same entry.
- **Expiry:** entries expire after `REVIEW_TTL`.
- **The cache is optional.** If Redis is down or returns a corrupt entry, the request still goes to the LLM, and the error is only logged. With no `REDIS_URL`, a no-op cache is used.

How to run

with Docker Compose

```
cp backend/.env.example backend/.env   # then set LLM_PROVIDER, LLM_API_KEY, LLM_MODEL
docker compose up --build
```

- Frontend: http://localhost:5173
- Backend: http://localhost:8080
- Compose starts Redis and points the backend at it, overriding `REDIS_URL` from `.env`.
- Inside a container, `localhost` is the container itself. To reach an LLM server on your machine (for example LM Studio), set `LLM_URL=http://host.docker.internal:1234/v1`.

on local development

```
# Backend
cd backend
cp .env.example .env    # set LLM_* values; leave REDIS_URL empty to run without a cache
go run .

# Frontend
cd frontend
cp .env.example .env    # VITE_API_URL=http://localhost:8080
npm install
npm run dev
```

Configuration

Backend settings come from environment variables, or from `backend/.env` when it exists.

Variable	Default	Description
`PORT`	`8080`	HTTP port
`CORS_ALLOWED_ORIGINS`	`http://localhost:5173`	Comma-separated list of allowed origins
`LLM_PROVIDER`	(required)	`openai` or `anthropic`
`LLM_API_KEY`		API key for the provider or gateway
`LLM_MODEL`	(required)	Model ID, for example `gpt-5.4` or `claude-sonnet-5-5`
`LLM_URL`	provider default	Custom base URL: an AI gateway, OpenRouter, LiteLLM or a local server
`LLM_MAX_TOKENS`	`2048`	Max output tokens. Raise it for reasoning models, whose thinking tokens count toward this limit
`REDIS_URL`	empty (no cache)	For example `redis://localhost:6379/0`
`REVIEW_TTL`	`1h`	How long a cached review is kept

The frontend reads `VITE_API_URL` at build time.

Variable	Default	Description
`VITE_API_URL`	(required)	Backend URL, for example `http://localhost:8080`

LLM Integration

`LLM_PROVIDER=openai` speaks the OpenAI Chat Completions API, so any OpenAI-compatible gateway works by changing only `LLM_URL`:

```
# OpenAI API Directly
LLM_PROVIDER=openai
LLM_MODEL=claude-sonnet-5-5

# Anthropic API Directly
LLM_PROVIDER=openai
LLM_MODEL=gpt-5.4
```

```
# AI Gateway or Specific Endpoint eg. OpenRouter

LLM_PROVIDER=openai
LLM_URL=https://openrouter.ai/api/v1
LLM_MODEL=anthropic/claude-sonnet-5-5

# Local model (LM Studio)

LLM_PROVIDER=openai
LLM_URL=http://localhost:1234/v1
LLM_MODEL=google/gemma-4-26b-a4b-it
```

Unit Testing

```
cd backend
go test ./...
```

The tests in `internal/review/review_test.go` use mocked LLM and cache clients. They cover:

- **Request handling:** bad requests, normalization and LLM errors.
- **Cache behavior:** cache hit and miss, and Redis failures.
- **Response parsing:** markdown fences, reasoning text with braces before the JSON, out-of-range lines, and invalid JSON.

Evaluation

Unit tests check the plumbing. The evaluation checks whether the AI's answers are actually right.

Dataset

`eval/dataset.json` has 14 cases: 10 in Python and 4 in Go.

Each case states the intent in a function name, docstring or comment, so "correct" is unambiguous. Bug cases are pure logic errors, meaning the code runs without crashing but returns a wrong result. Crashes and lint-level mistakes are left out on purpose, because a test run or linter already catches them.

Group	Cases
Logic errors	off-by-one, wrong/inverted condition, operator precedence, unreachable branch, wrong initial value, integer division, early return, mutation during iteration

Group	Cases
Multi-bug	two independent bugs on adjacent lines (`go-merge-intervals-two-bugs`)
Clean code	1 correct snippet (binary search); any comment is a false positive
Prompt injection	a comment that tells the reviewer to return no issues

Each case lists `expected_bugs[].lines`: every line where pointing at the bug counts as correct. A bug can often be pointed at from several places.

Scoring

Each case was run through the app, and the answers were scored by hand in a spreadsheet

Column	Meaning
`bugs_found`	Expected bugs whose lines fall inside a reported `startLine..endLine`
`false_positives`	Comments that match no expected bug
`explanation_score`	0 = wrong, 1 = right but vague, 2 = clear and says when it fails
`fix_correct`	Correct / Partial / Wrong
`passed`	`find_bugs`: every expected bug found. `no_issues`: zero comments.

Line numbers alone are not enough. A reported range can cover a bug's line without the explanation or fix actually addressing that bug, so a person reads every explanation and fix.

Results: `google/gemma-4-26b-a4b-it` (14 cases, Python and Go)

https://docs.google.com/spreadsheets/d/1BCBOm8HsJMPF0sTlaKf0nPHbw4orkRd77ipuUvyq8rg/edit?usp=sharing

Metric	Result
Cases passed	13 / 14
Bugs found	13 / 14 (93%)
False positives	0 (including the clean case)
Prompt injection	Ignored
Explanation score	1.9 / 2 on average
Fix judged correct	12 / 13

Metric	Result
Latency	324 ms – 1.1 s (median ~0.8 s)
Tokens per request	~870 – 1,270 in, ~70 – 150 out

Findings

1. **Detection is strong; patches are weaker.** Gemma found almost every bug and explained it well, but 4 of the fixes do not apply cleanly as patches:
   - leap year: the fix drops the `def` line
   - contains: the fix leaves an extra `}`
   - inverted error check: 3 lines replace a 2-line range
   - shipping tiers: wrong indentation

   The fix logic was right; the fix just did not match the reported range or the indentation.
2. **Multi-bug cases are the limit for the small model.** On `go-merge-intervals-two-bugs`:
   - Gemma found only the first bug.
   - Its fix covered one line of a two-line range, so applying it deletes line 15.

   With the same prompt, Claude found both bugs as separate entries with exact ranges and correct tab indentation. So the prompt works, and the gap comes from model capacity.

Evaluation limitations

- **Coverage:** 14 cases, Python and Go only. The app also accepts TypeScript, JavaScript and Java, but those were not evaluated.
- **Runs and raters:** one main model, mostly one run per case, and one human rater.